PoSt (Proof-of-Space-time) 개념과 구현

개요

PoSt (Proof-of-Space-time)는 파일코인 네트워크에서 저장 제공자(Storage Provider)가 약속된 기간 동안 데이터를 지속적으로 저장하고 있음을 증명하는 암호학적 메커니즘입니다. PoRep이 데이터의 복제를 증명한다면, PoSt는 시간의 경과에 따른 지속적인 저장을 증명합니다.

파일코인에서는 두 가지 유형의 PoSt를 사용합니다:

- **Winning PoSt**: 블록 생성 권한을 얻기 위한 증명
- **Window PoSt**: 지속적인 저장을 증명하기 위한 정기적 증명

Winning PoSt vs Window PoSt

Winning PoSt

Winning PoSt는 저장 제공자가 새로운 블록을 생성할 권한을 얻기 위해 제출하는 증명입니다.

특징:

- **목적**: 블록 생성 권한 획득 및 블록 보상 수령
- **빈도**: 매 에포크(30초)마다 기회 제공
- **대상 섹터**: 무작위로 선택된 소수의 섹터 (일반적으로 1개)
- **시간 제약**: 매우 짧은 시간 내에 증명 생성 필요
- **실패 시 패널티**: 블록 생성 기회 상실 (저장 파워 손실 없음)

매개변수 특성:

```
// Winning PoSt의 매개변수 계산
pub fn winning_post_setup_params(post_config: &PoStConfig) -> Result<WinningPo
    // 섹터 수로 challenge 수를 나누어 섹터당 challenge 수 계산
    let param_sector_count = post_config.challenge_count / post_config.sect
    let param_challenge_count = post_config.challenge_count / param_sector_
    Ok(fallback::SetupParams {
        sector_size: post_config.padded_sector_size().into(),
        challenge_count: param_challenge_count,
        sector_count: param_sector_count,
        api_version: post_config.api_version,
    })
}
```

Window PoSt

Window PoSt는 저장 제공자가 약속된 데이터를 지속적으로 저장하고 있음을 정기적으로 증명하는 메커니즘입니다.

특징:

- **목적**: 지속적인 데이터 저장 증명 및 저장 파워 유지
- **빈도**: 24시간마다 한 번 (proving period)
- **대상 섹터**: 모든 활성 섹터를 파티션으로 나누어 증명
- **시간 제약**: 30분의 증명 윈도우 제공
- **실패 시 패널티**: 저장 파워 감소 및 슬래싱 위험

매개변수 특성:

```
// Window PoSt의 매개변수는 직접 사용
pub fn window_post_setup_params(post_config: &PoStConfig) -> WindowPostSetupPa
        fallback::SetupParams {
                sector_size: post_config.padded_sector_size().into(),
                challenge_count: post_config.challenge_count, // 직접 사용
                sector_count: post_config.sector_count, // 직접 사용
                api_version: post_config.api_version,
        }
}
```

Fallback PoSt 메커니즘

파일코인의 PoSt 구현은 "Fallback PoSt"라고 불리는 메커니즘을 사용합니다. 이는 원래 계획된 PoSt 방식에서 실용적인 대안으로 채택된 방식입니다.

핵심 구조

```
pub struct FallbackPoSt<'a, Tree>
where
Tree: MerkleTreeTrait,
{
        _t: PhantomData<&'a Tree>,
```

```
}
```

공개 매개변수

```
pub struct PublicParams {
    /// 섹터 크기 (바이트)
    pub sector_size: u64,
    /// 섹터당 challenge 수
    pub challenge_count: usize,
    /// 증명 대상 섹터 수
    pub sector_count: usize,
    pub api_version: ApiVersion,
}
```

증명 구조

```
pub struct Proof<P: MerkleProofTrait> {
    pub sectors: Vec<SectorProof<P>>,
    }

    pub struct SectorProof<Proof: MerkleProofTrait> {
    /// 각 challenge에 대한 머클 증명들
    pub inclusion_proofs: Vec<MerkleProof<...>>,
```

```
        /// 섹터의 CommC (원본 데이터 커밋먼트)
        pub comm_c: <Proof::Hasher as Hasher>::Domain,
        /// 섹터의 CommRLast (마지막 레이어 커밋먼트)
        pub comm_r_last: <Proof::Hasher as Hasher>::Domain,
}
```

Challenge 생성 과정

PoSt의 핵심은 무작위성을 기반으로 한 challenge 생성입니다. 이 과정은 두 단계로 구성됩니다:

1. 섹터 Challenge 생성

먼저 어떤 섹터들이 증명 대상이 될지 결정합니다:

```
pub fn generate_sector_challenge<T: Domain>(
randomness: T, // 체인에서 제공하는 무작위성
n: usize, // challenge 인덱스
sector_set_len: u64, // 전체 섹터 집합 크기
prover_id: T, // 증명자 ID
) -> Result<u64> {
        let mut hasher = Sha256::new();
        hasher.update(AsRef::<[u8]>::as_ref(&prover_id));
        hasher.update(AsRef::<[u8]>::as_ref(&randomness));
        hasher.update(&n.to_le_bytes()[..]);
```

```
        let hash = hasher.finalize();

        let sector_challenge = LittleEndian::read_u64(&hash[..8]);

        let sector_index = sector_challenge % sector_set_len;



        Ok(sector_index)

}
```

과정 설명:

1. 증명자 ID, 무작위성, challenge 인덱스를 SHA256으로 해시
2. 해시 결과의 첫 8바이트를 64비트 정수로 변환
3. 전체 섹터 수로 모듈로 연산하여 섹터 인덱스 결정

2. 리프 Challenge 생성

선택된 섹터 내에서 어떤 데이터 블록들을 증명할지 결정합니다:

```
pub fn generate_leaf_challenge<T: Domain>(

pub_params: &PublicParams,

randomness: T,

sector_id: u64,

leaf_challenge_index: u64,

) -> u64 {

        let mut hasher = Sha256::new();

        hasher.update(AsRef::<[u8]>::as_ref(&randomness));

        hasher.update(&sector_id.to_le_bytes()[..]);
```

```
        hasher.update(&leaf_challenge_index.to_le_bytes()[..]);

        let  hash = hasher.finalize();

        let  leaf_challenge = LittleEndian::read_u64(&hash[..8]);



        leaf_challenge % (pub_params.sector_size / NODE_SIZE as u64)

}
```

과정 설명:

1. 무작위성, 섹터 ID, challenge 인덱스를 SHA256으로 해시
2. 해시 결과를 섹터 내 노드 수로 모듈로 연산
3. 결과로 나온 인덱스가 증명해야 할 리프 노드

섹터별 Challenge 인덱스 계산

API 버전에 따라 challenge 인덱스 계산 방식이 다릅니다:

```
pub  fn  get_challenge_index(

api_version: ApiVersion,

sector_index: usize,

challenge_count_per_sector: usize,

challenge_index: usize,

) -> u64 {

        (match  api_version {

                // V1.0.0, V1.1.0: 섹터별로 순차적 인덱스

                ApiVersion::V1_0_0 | ApiVersion::V1_1_0 => {
```

```
                sector_index * challenge_count_per_sector + challenge_index
                }
                // V1.2.0: 글로벌 challenge 인덱스 사용
                ApiVersion::V1_2_0 => challenge_index,
        } as u64)
}
```

버전별 차이점:

V1.0.0 / V1.1.0

- 각 섹터마다 독립적인 challenge 인덱스 범위 사용
- 섹터 0: challenge 0~N-1, 섹터 1: challenge N~2N-1, ...
- 섹터 간 challenge 중복 방지

V1.2.0

- 모든 섹터가 동일한 글로벌 challenge 인덱스 사용
- 동일한 challenge 인덱스로 모든 섹터의 동일한 상대적 위치 증명
- 더 효율적인 배치 처리 가능

증명 생성 과정

1. Vanilla 증명 생성

```
pub fn vanilla_proof<Tree: MerkleTreeTrait>(
sector_id: SectorId,
```

```
priv_inputs: &PrivateInputs<'_, Tree>,
challenges: &[u64],
) -> Result<Proof<Tree::Proof>> {
let priv_sector = &priv_inputs.sectors[0];
let tree = priv_sector.tree;
// 각 challenge에 대한 머클 증명 생성
let inclusion_proofs = (0..challenges.len())
.into_par_iter()
.map(|challenged_leaf_index| {
let challenged_leaf = challenges[challenged_leaf_index];
let proof = tree.gen_cached_proof(
challenged_leaf as usize,
Some(rows_to_discard)
)?;
// 증명 유효성 검증
ensure!(
proof.validate(challenged_leaf as usize) &&
proof.root() == priv_sector.comm_r_last,
"Generated vanilla proof for sector {} is invalid",
sector_id
);
Ok(proof)
})
```

```
.collect::<Result<Vec<_>>>()?;


Ok(Proof {
sectors: vec![SectorProof {
inclusion_proofs,
comm_c: priv_sector.comm_c,
comm_r_last: priv_sector.comm_r_last,
}],
})
}
```

2. 배치 증명 생성

실제 구현에서는 여러 섹터를 동시에 처리합니다:

```
// 파티션별로 섹터들을 그룹화
for (j, (pub_sectors_chunk, priv_sectors_chunk)) in  pub_inputs
.sectors
.chunks(num_sectors_per_chunk)
.zip(priv_inputs.sectors.chunks(num_sectors_per_chunk))
.enumerate()
{
// 각 섹터에 대해 병렬로 증명 생성
let (proofs, faults) = pub_sectors_chunk
```

```
.par_iter()
.zip(priv_sectors_chunk.par_iter())
.enumerate()
.map(|(i, (pub_sector, priv_sector))| {
// challenge 생성 및 증명 생성
// ...
})
.collect();
}
```

증명 검증 과정

1. 기본 검증

```
pub fn verify(
pub_params: &Self::PublicParams,
pub_inputs: &Self::PublicInputs,
partition_proof: &Self::Proof,
) -> Result<bool> {
// 각 섹터 증명 검증
let is_valid = pub_sectors_chunk
.par_iter()
.zip(proof.sectors.par_iter())
```

```
.enumerate()
.map(|(i, (pub_sector, sector_proof))| {
// 1. CommR 검증: H(CommC || CommRLast) == CommR
let comm_r_last = inclusion_proofs[0].root();
if hash2(&comm_c, &comm_r_last) != pub_sector.comm_r {
return Ok(false);
}
// 2. 각 inclusion proof 검증
inclusion_proofs
.par_iter()
.enumerate()
.map(|(n, inclusion_proof)| {
// challenge 재계산
let challenge_index = get_challenge_index(...);
let challenged_leaf = generate_leaf_challenge_inner(...);
// 머클 증명 검증
inclusion_proof.validate(challenged_leaf as usize)
})
.all(|v| v)
})
.all(|v| v?);
Ok(is_valid)
```

```
}
```

2. 검증 단계

1. **CommR 일치성 검증**: `H(CommC || CommRLast) == CommR`
2. **Challenge 재계산**: 동일한 알고리즘으로 challenge 재생성
3. **머클 증명 검증**: 각 inclusion proof의 유효성 확인
4. **경로 길이 검증**: 예상되는 머클 트리 깊이와 일치 확인

성능 최적화

1. 병렬 처리

```
// 섹터별 병렬 처리
.par_iter()
.zip(priv_sectors_chunk.par_iter())
.enumerate()
.map(|(i, (pub_sector, priv_sector))| {
// 각 섹터의 증명을 병렬로 생성
})

// Challenge별 병렬 처리
(0..pub_params.challenge_count)
.into_par_iter()
.fold(|| (Vec::new(), BTreeSet::new()), |acc, n| {
```

```
// 각 challenge를 병렬로 처리

})
```

2. 캐시 활용

- **머클 트리 캐싱**: `gen_cached_proof` 로 중간 노드 재사용
- **Challenge 해시 재사용**: 고정 입력에 대한 해시 결과 캐싱
- **매개변수 캐싱**: 공개 매개변수와 검증 키 캐싱

3. 메모리 최적화

- **스트리밍 처리**: 대용량 섹터를 청크 단위로 처리
- **지연 로딩**: 필요한 시점에만 머클 트리 로드
- **가비지 컬렉션**: 사용 완료된 증명 데이터 즉시 해제

보안 고려사항

1. 무작위성 보장

- **체인 무작위성**: 블록체인에서 제공하는 검증 가능한 무작위성 사용
- **예측 불가능성**: 미래의 challenge를 미리 예측할 수 없도록 설계
- **조작 방지**: 증명자가 challenge 생성에 영향을 줄 수 없도록 보장

2. 시간-공간 트레이드오프 방어

- **충분한 Challenge 수**: 통계적으로 충분한 샘플링으로 부정행위 탐지
- **무작위 샘플링**: 예측 불가능한 위치의 데이터 요구

- **시간 제약**: 실시간 데이터 접근을 강제하는 시간 제한

3. 결함 섹터 처리

```
// 결함 섹터 탐지 및 보고
let mut faulty_sectors = BTreeSet::new();

match proof {
        Ok(proof) => {
                if !proof.validate(...) {
                        error!("Found faulty sector: {:?}", sector_id);
                        faults.insert(sector_id);
                }
        }
        Err(err) => {
                error!("Found faulty sector: {:?}: {:?}", sector_id, err);
                faults.insert(sector_id);
        }
}

if !faulty_sectors.is_empty() {
        return Err(Error::FaultySectors(faulty_sectors.into_iter().collect()));
}
```

실제 사용 예시

Winning PoSt 생성

```
use filecoin_proofs::generate_winning_post;

let  proof = generate_winning_post::<Tree>(
        &post_config, // PoSt 설정
        &randomness, // 체인 무작위성
        &replicas, // 섹터 복제본들
        prover_id, // 증명자 ID
)?;
```

Window PoSt 생성

```
use filecoin_proofs::generate_window_post;

let  proof = generate_window_post::<Tree>(
        &post_config, // PoSt 설정
        &randomness, // 체인 무작위성
        &replicas, // 모든 활성 섹터들
        prover_id, // 증명자 ID
)?;
```

결론

PoSt는 파일코인 네트워크의 핵심 보안 메커니즘으로, 저장 제공자들이 약속된 데이터를 지속적으로 저장하고 있음을 효율적으로 증명할 수 있게 합니다. Winning PoSt와 Window PoSt의 이중 구조를 통해 블록 생성 권한과 저장 파워 유지를 분리하여 관리하며, Fallback PoSt 메커니즘을 통해 실용적이고 확장 가능한 구현을 제공합니다.

무작위 challenge 생성, 병렬 처리, 그리고 강력한 보안 모델을 통해 파일코인 네트워크의 신뢰성과 효율성을 보장하는 중요한 역할을 수행합니다.